NumPy: numerical computations

Setup and arrays: `import numpy as np` imports NumPy. `np.array([1., 2.])` is a vector array of shape `(2,)`; `np.array([[1., 2.], [3., 4.]])` is a matrix of shape `(2, 2)`. `np.ones(n)` creates a length-n array of ones.

Products: `A @ x` computes $A\mathbf{x}$. If `A` has shape `(m, n)`, use `x` of shape `(n,)`; the output has shape `(m,)`. `A * x` uses entrywise arithmetic. `A @ (x + t*z)` tests how a change in the input affects its output.

Solve a system: for square, nonsingular `A` and a compatible right-hand side `b`,

```
x = np.linalg.solve(A, b)
print(x)
print(np.allclose(A @ x, b))
```

`np.linalg.solve(A, b)` returns the numerical solution to $A\mathbf{x} = \mathbf{b}$. `print(...)` displays a value; `np.allclose(...)` returns one Boolean checking agreement within tolerances. It is not an exact equality test. `solve` does not handle a rectangular or singular coefficient matrix.

Rank: `np.linalg.matrix_rank(A)` returns a numerical estimate of rank. For an $m \times n$ matrix of rank r, there are r independent columns and nullity $n - r$. Floating-point rank decisions use a tolerance.

Determinant: `np.linalg.det(A)` computes the determinant of a square matrix. `float(...)` converts a scalar result to a Python float. Exactly, $\det(A) \neq 0$ means A is invertible; numerical output may contain roundoff, so a tiny reported value alone is not an exact singularity test.

SymPy: exact matrices

Setup: SymPy supports symbolic mathematics and exact arithmetic; NumPy is designed for numerical array computations. SymPy can, for instance, keep fractions exact during row reduction. Import it with `import sympy as sp` and use integer matrix entries to retain exact arithmetic.

Matrices and vectors: `sp.Matrix([[1, 2], [0, 1]])` creates a 2×2 matrix; `sp.Matrix([3, 4])` creates a 2×1 *column* matrix. `sp.ones(n, 1)` creates a column of n ones; `sp.eye(n)` creates I_n.

Shape and transpose: `z.shape` reports (rows, columns). For an $n \times 1$ column, `z.T` is a $1 \times n$ row.

Read row-reduction output

Two returned objects: `R, pivots = M.rref()` assigns the RREF to `R` and its pivot-column indices to the tuple `pivots`. Indices start at zero: `(0, 2)` means mathematical columns 1 and 3.

Augmented matrices: interpret `M` as $[A \mid \mathbf{b}]$ when solving a system. The final column is the right-hand side. A pivot in that column means inconsistency; otherwise nonpivot variable columns are free.

Indexing and original pivot columns

Select an entry: rows and columns start at index zero. `A[i, j]` selects one entry; `A[0, 2]` is the entry in the first row and third column.

Slices: `A[:, j]` takes all rows of column `j`, producing a column matrix. `A[j, :]` takes row `j`. The colon means "all."

```
R, pivots = A.rref()
columns = [A[:, j] for j in pivots]
```

List comprehension: for each `j` in `pivots`, evaluate `A[:, j]` and collect it in a list. The result is a basis for col(A). Use the original `A`, not `R`. Do not add one to indices used in Python.

Equivalent loop: `[]` starts an empty list; `append` adds one item. Indentation marks the loop body.

```
columns = []
for j in pivots:
    columns.append(A[:, j])
```

Null-space bases and directions

Multiplication: with two numbers, `*` is ordinary multiplication; with a number and a matrix, it scales every entry. With two SymPy matrices, `*` is matrix multiplication, provided the inner dimensions match. Thus `A * z` computes $A\mathbf{z}$ for a column matrix `z`. NumPy arrays use `@` for this product and `*` for entry-wise arithmetic. `z.T * A` has a different meaning and need not be zero when `A * z` is zero.

Basis list: `basis = A.nullspace()` returns a list of SymPy column matrices forming a basis for null(A). `len(basis)` is the nullity. `basis[0]` selects the first vector only when the list is nonempty:

```
z = basis[0]
A * z
```

The product is a zero column. `A * basis` is an error: `basis` is a list, not one vector. If the null space is $\{\mathbf{0}\}$, `basis` is `[]`; `basis[0]` then raises an error. For $A\mathbf{z} = \mathbf{0}$, $A(\mathbf{x} + t\mathbf{z}) = A\mathbf{x}$ for every scalar t.

Augment with an identity matrix

Join columns: `M = A.row_join(sp.eye(n))` forms $[A \mid I_n]$ for $n \times n$ `A`.

`R, pivots = M.rref()` gives its RREF. If A is invertible, the result is $[I_n \mid A^{-1}]$; `R[:, n:]` selects the inverse block (all rows, columns from index n onward).